Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Программно-аппаратные средства защиты информации»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Регистрация пользователя и модели разграничения доступа. Вариант 2»

Выполнили:
Шибаев Иван Дмитриевич, студент группы N3352

_____
(подпись)

Малышев Григорий Игоревич, студент группы N3352

_____
(подпись)

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Введение 3

1 Архитектура и структура данных 4

1.1 Взаимодействие компонентов 4

1.2 Таблицы и миграция 4

2 Регистрация и аутентификация 6

2.1 Валидация и создание учётной записи 6

2.2 Хэширование паролей 7

2.3 Вход и определение пользователя 8

3 Дискреционная модель доступа 9

3.1 Правило принятия решения 9

3.2 Выдача и отзыв прав 9

4 Проверки и результаты 13

4.1 Автоматические тесты backend 13

4.2 Браузерная проверка 13

5 Воспроизведение и материалы сдачи 15

5.1 Запуск и проверка 15

5.2 Исходники и состояние Git 15

6 Безопасность и ограничения 17

Заключение 18

Список использованных источников 19

ВВЕДЕНИЕ

Цель работы — разработать веб-приложение с регистрацией пользователя, сохранением хэша пароля в PostgreSQL и серверной проверкой прав доступа к защищаемым ресурсам. По варианту 2 реализована дискреционная модель: у каждого ресурса есть владелец, который выдаёт другим пользователям отдельные права на чтение, изменение и удаление [1].

Разработанное приложение «Контур» включает форму регистрации, вход, работу с документами и управление правами. Решение запускается через Docker Compose. Проверки выполнены на собственном локальном контейнерном стенде с настоящим PostgreSQL; в отчёте используются результаты pytest, браузерного сценария и снимки работающего интерфейса.

1 Архитектура и структура данных

1.1 Взаимодействие компонентов

Состав сервисов показан на рисунке 1. Frontend написан на React и TypeScript. Vite собирает статические файлы, которые Nginx передаёт браузеру. Запросы с префиксом «/api» Nginx перенаправляет в FastAPI. Backend использует SQLAlchemy и драйвер psycopg для обращения к PostgreSQL.

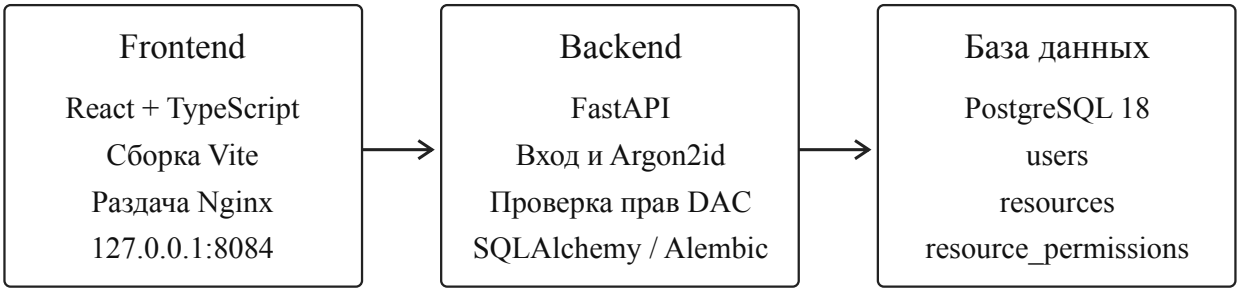


Рисунок 1 — Взаимодействие сервисов Docker Compose

Для frontend и backend подготовлены отдельные Dockerfile. PostgreSQL использует именованный том, поэтому остановка и повторный запуск контейнеров сохраняют данные. На хост опубликован только адрес 127.0.0.1:8084. Порты backend и БД доступны внутри сети Compose.

Перед запуском backend проверяется готовность БД. Перед запуском frontend проверяется готовность backend, включая запрос к PostgreSQL. Использована зависимость с условием «service_healthy», поскольку само создание контейнера ещё не означает готовность сервиса [2].

1.2 Таблицы и миграция

Схема содержит три прикладные таблицы, назначение которых приведено в таблице 1. Дополнительно Alembic хранит номер применённой миграции в служебной таблице.

Таблица 1 — Основные отношения базы данных

Таблица	**Состав и ограничения**
users	id, уникальный login, password_hash, created_at. Открытый пароль отсутствует.
resources	id, owner_id, title, content, created_at. Владелец ссылается на users.
resource_permissions	user_id, resource_id, can_read, can_update, can_delete. Пара идентификаторов — первичный ключ.

Внешние ключи связывают назначение с существующими пользователем и ресурсом. При удалении ресурса его назначения удаляются каскадно. Составной ключ исключает несколько противоречивых записей для одной пары «пользователь — ресурс».

Миграция «0001_registration_dac» создаёт таблицы, уникальное ограничение логина, внешние ключи и индекс владельца. При запуске выполняется команда применения миграций. Такой подход сохраняет историю схемы и позволяет проверить её откат [3]. Тестовый стенд действительно выполняет применение, сравнение с ORM, откат и повторное применение миграции в отдельной схеме PostgreSQL.

2 Регистрация и аутентификация

2.1 Валидация и создание учётной записи

Страница «/register» содержит поля login и password. Форма отправляет JSON-запрос POST «/api/register». Backend проверяет данные и создаёт строку users. Коды результатов приведены в таблице 2.

Таблица 2 — Ответы регистрации

Событие	**HTTP**	**Результат**
Корректная регистрация	201	Пользователь создан
Логин уже занят	409	Понятный отказ
Ошибка входных данных	422	Сообщение о требованиях

Логин содержит от 3 до 32 символов: латинские буквы, цифры, точку, подчёркивание и дефис. Регистр значим. Пароль содержит от 8 до 128 символов, хотя бы одну заглавную и строчную латинскую букву, цифру и ASCII-знак пунктуации. Подчёркивание также считается специальным символом. Верхняя граница длины ограничивает объём обрабатываемых входных данных.

Уникальность обеспечивается ограничением PostgreSQL. Приложение обрабатывает конфликт INSERT и возвращает 409; это работает и при одновременных попытках зарегистрировать одинаковый логин. Ошибки валидации возвращаются без исходного значения поля, чтобы пароль не оказался в ответе вместе с описанием ошибки.

На рисунке 2 показаны форма и фактический ответ успешной регистрации. Снимки повторной регистрации и слабого пароля также сохранены в каталоге screenshots.

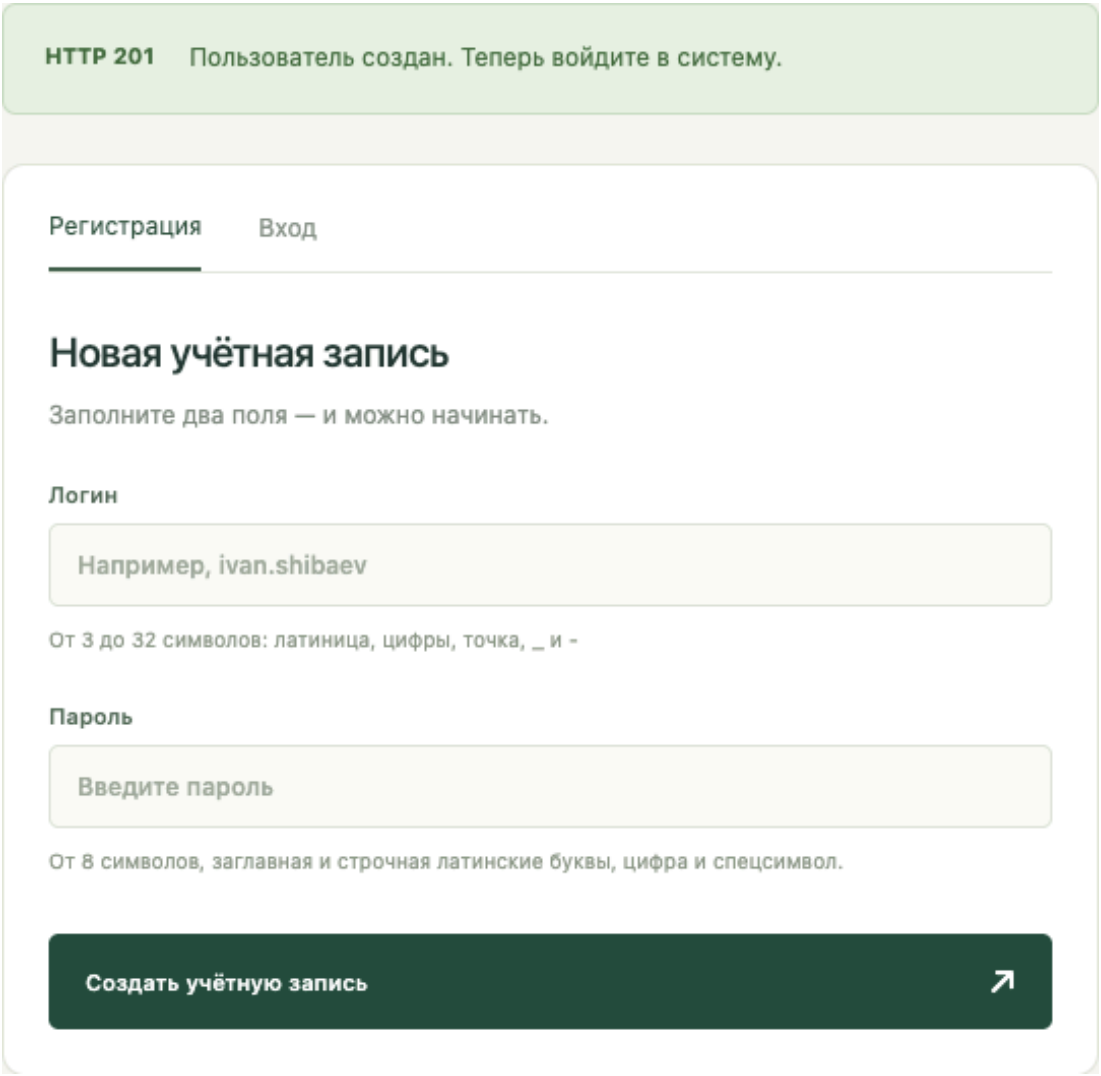


Рисунок 2 — Форма регистрации и сообщение об успешном создании пользователя

2.2 Хэширование паролей

Используется Argon2id через argon2-cffi. Для каждого пароля библиотека генерирует случайную соль. В строке хэша сохраняются алгоритм, версия, параметры, соль и результат вычисления, необходимые для последующей проверки [4].

В выполненном эксперименте заданы следующие параметры:

– ARGON2_TIME_COST — 3;
– ARGON2_MEMORY_COST — 65536 КиБ;
– ARGON2_PARALLELISM — 4;
– длина соли — 16 байт, длина хэша — 32 байта.

Настройки времени, памяти и параллелизма вынесены в окружение. HASH_SCHEME допускает только значение «argon2id». Тест проверяет префикс и параметры сохранённого хэша, успешную проверку исходного пароля и различие хэшей одинаковых паролей двух пользователей благодаря отдельным солям. Сведения о формате хэшей из работающей БД сохранены в файле «verification/database.txt» без открытых паролей.

2.3 Вход и определение пользователя

POST «/api/login» проверяет пароль по хэшу и выдаёт подписанный JWT с идентификатором пользователя и сроком действия 30 минут. Backend проверяет подпись HS256, срок, издателя и получателя токена. Применение JWT для передачи подтверждённой личности и отдельная проверка пароля соответствуют описанному в документации FastAPI подходу [5].

Токен передаётся в заголовке Authorization. Frontend хранит его только в памяти вкладки. При выходе или перезагрузке страницы требуется новый вход. Идентификатор владельца создаваемого ресурса определяется по проверенному токену; клиент не может передать свой owner_id. Права в токен не включаются: их актуальное состояние проверяется по БД.

3 Дискреционная модель доступа

3.1 Правило принятия решения

Владелец обладает всеми тремя правами на собственный ресурс. Для другого пользователя backend находит запись resource_permissions и проверяет флаг, соответствующий запрошенной операции. При отсутствии записи или выключенном флаге возвращается 403. Без действительного токена возвращается 401.

Права независимы. Чтение не разрешает изменение или удаление; право изменения само по себе не открывает содержимое. Выдавать и отзывать права может только владелец. Даже пользователь со всеми тремя назначенными правами не получает возможность дальнейшего делегирования.

Список доступных объектов содержит метаданные: идентификатор, название, владельца и текущие флаги. Содержимое документа выдаётся только отдельным запросом с проверкой чтения. Название и идентификатор считаются метаданными. Ввод известного идентификатора напрямую не обходит проверку.

3.2 Выдача и отзыв прав

В интерфейсе владелец выбирает ресурс, вводит логин зарегистрированного участника и отмечает разрешённые действия. PUT сохраняет полный набор флагов для пары «участник — ресурс»; повторный запрос обновляет существующую запись. Неуказанные флаги получают значение false. Кнопка «Отозвать» удаляет назначение.

В фактическом браузерном опыте владелец ivan.mw3jnfxi создал ресурс № 4 «План лабораторной работы» и выдал пользователю grigory.mw3jnfxi только право чтения. Сохранённое назначение показано на рисунке 3.

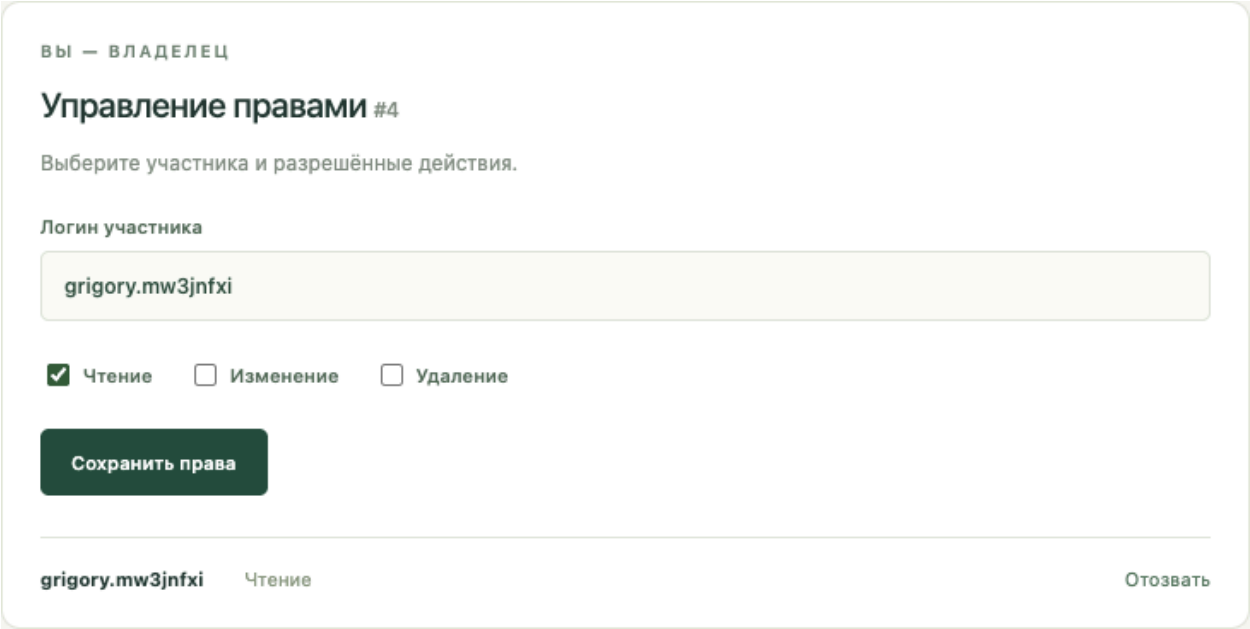


Рисунок 3 — Назначение отдельного права чтения владельцем ресурса

Получатель вошёл под собственной учётной записью и прочитал содержимое; сервер ответил 200. Результат показан на рисунке 4. Попытка сохранить изменения с тем же токеном вернула 403.

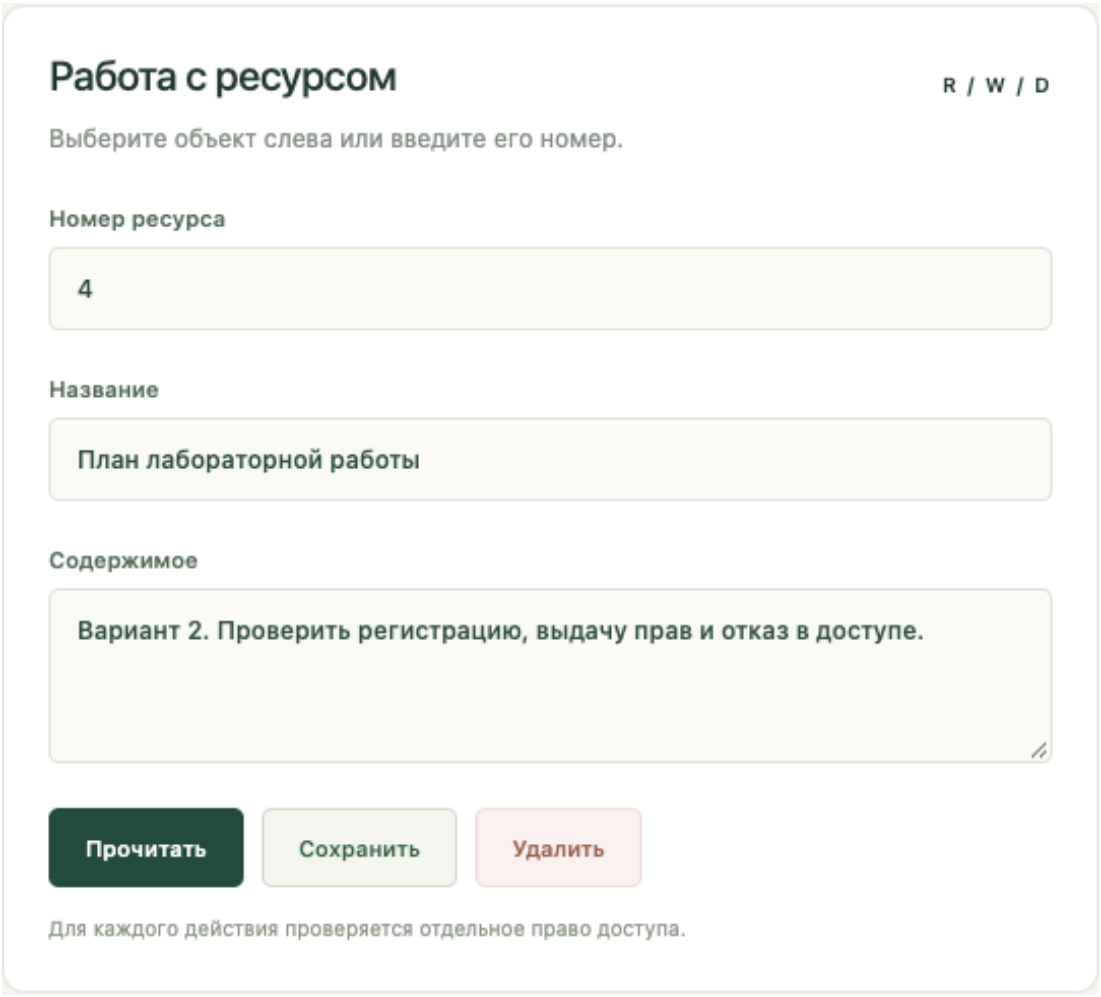


Рисунок 4 — Открытие содержимого пользователем с правом чтения

Третья учётная запись observer.mw3jnfxi не получила прав на ресурс. Пользователь ввёл его известный номер, однако содержимое осталось пустым, а сервер вернул 403. На рисунке 5 показаны две области одной страницы: сообщение сервера и карточка ресурса.

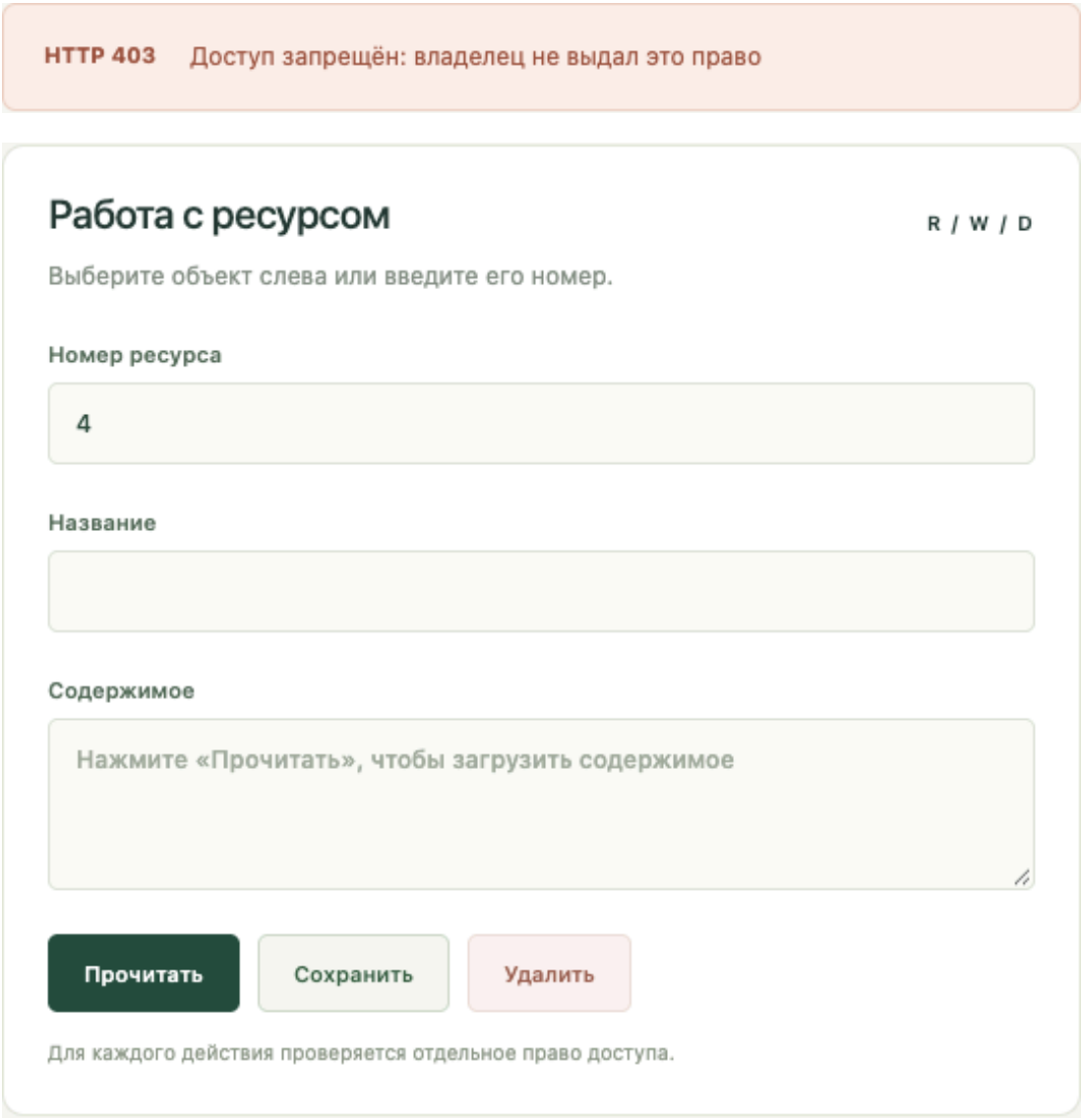


Рисунок 5 — Отказ в чтении пользователю без назначенных прав

После отзыва права владельцем получатель повторил чтение с прежним действительным токеном и получил 403. Это подтверждает проверку актуальных назначений на backend. Затем владелец восстановил чтение, оставив на стенде демонстрационный пример.

4 Проверки и результаты

4.1 Автоматические тесты backend

Выполнено 30 тестов pytest, ошибок и пропусков нет. Проверки работают с настоящим PostgreSQL в случайной схеме с префиксом «lab4_test_». После завершения удаляется только эта схема; пользовательские данные приложения не очищаются. Результат сохранён в JUnit XML.

Проверки охватывают:

- успешную регистрацию, формат Argon2id и случайную соль;
- занятый логин, в том числе конкурентную регистрацию;
- слабые пароли и недопустимые логины;
- отсутствие паролей в логах и ответах ошибок;
- вход, обязательную аутентификацию, поддельные и истёкшие JWT;
- доступ владельца, разрешённое чтение и отказ постороннему;
- независимость чтения, изменения и удаления;
- запрет выдачи и отзыва прав не владельцем;
- немедленное применение отзыва к следующему запросу;
- обновление назначения без дубликатов и каскадное удаление;
- запрет подмены владельца и отсутствие содержимого в списке метаданных.

4.2 Браузерная проверка

Через Google Chrome 157.0.8103.38 выполнено 14 проверок. Сценарий использует настоящие формы и кнопки, три самостоятельных сеанса и действующий backend. Ошибок JavaScript не обнаружено. На экране шириной 390 пикселей дополнительно проверено отсутствие горизонтального переполнения.

Основные фактические результаты сведены в таблицу 3. Статусы записаны из ответов работающего приложения, а не подставлены в интерфейс.

Таблица 3 — Результаты браузерного сценария

Проверенное действие	**HTTP**
Регистрация каждой из трёх учётных записей	201
Повторная регистрация занятого логина	409
Регистрация со слабым паролем	422
Выдача чтения владельцем	200
Чтение получателем права	200
Изменение при наличии только чтения	403
Чтение посторонним по известному ID	403
Отзыв назначения владельцем	204
Повторное чтение после отзыва	403

События приложения записываются в JSON с уровнями INFO и ERROR. Успешная регистрация и конфликт логина относятся к INFO, ошибка валидации — к ERROR. Логи содержат тип события и безопасные идентификаторы, без тела запроса, пароля, хэша и JWT. Сохранённые логи позволяют сопоставить регистрацию, изменение прав и отказы в доступе.

5 Воспроизведение и материалы сдачи

5.1 Запуск и проверка

Из каталога лабораторной выполняются следующие команды:

```
python3 scripts/init_env.py
docker compose up --build -d --wait
make check
```

Создание окружения генерирует случайные локальные секреты и сохраняет их в файле .env с правами 0600. Повторный запуск не перезаписывает конфигурацию. В проект и архив включён только образец .env.example. После подготовки конфигурации приложение запускается командой «docker compose up».

В README приведены структура проекта, переменные окружения, API, curl-примеры и последовательность демонстрации. Команда «make shots» выполняет браузерный сценарий при установленном Node.js и Google Chrome. Команды «make report» и «make archive» собирают PDF и архив для сдачи.

5.2 Исходники и состояние Git

Работа находится в каталоге «subjects/paszi/lab4». Сохранены исходники backend и frontend, миграция, тесты, lock-файл npm, закреплённые зависимости Python, Dockerfile, Compose, отчёт и результаты проверки. Исходное задание сохранено без изменений.

На рисунке 6 зафиксировано настоящее меню веток WebStorm: текущая ветка — main. При подготовке работы новые коммиты и публикация репозитория не выполнялись. Архив содержит проект и общий титульный шаблон с сохранением относительной структуры путей.

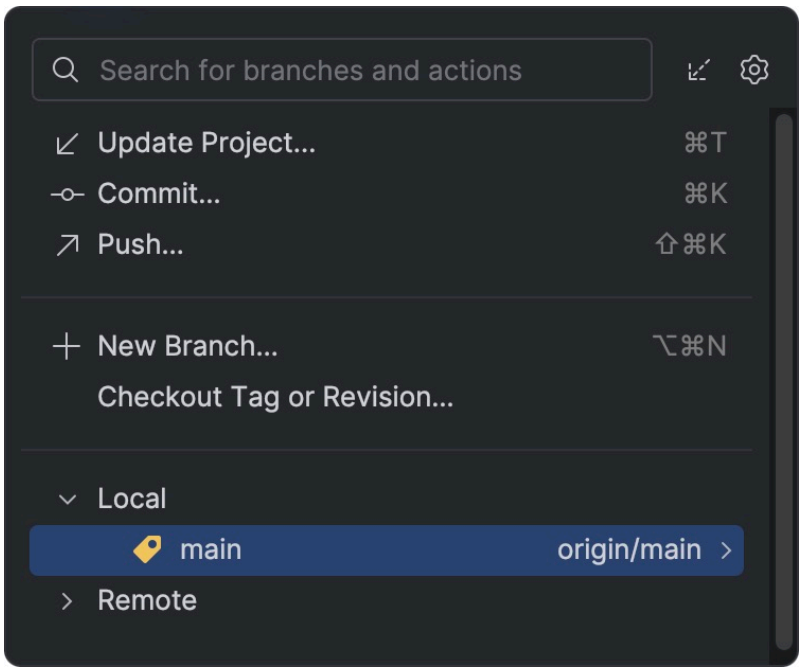


Рисунок 6 — Текущая ветка проекта в WebStorm

6 Безопасность и ограничения

На backend реализованы валидация, хэширование с солью, уникальность логина, аутентификация и проверка каждой операции с ресурсом. Пользовательский текст выводится средствами React без интерпретации HTML. Nginx задаёт Content Security Policy и запрещает определение типа содержимого по догадке. Backend работает от непривилегированного пользователя контейнера.

Учебный стенд использует HTTP на loopback. Для сетевого размещения нужны HTTPS, ограничение частоты запросов и отдельная эксплуатационная конфигурация. Нет подтверждения почты, восстановления пароля, MFA и централизованного неизменяемого журнала. Начальная роль PostgreSQL обладает административными правами; её следует отделить от роли приложения при эксплуатации.

Выход очищает токен из вкладки, но не отзывает уже скопированный JWT на сервере. Он сохраняет силу до истечения срока. Отзыв права блокирует следующие обращения, однако не стирает данные, прочитанные ранее. Передача владельца и право дальнейшего делегирования не реализованы. Пользователь с правом удаления может окончательно удалить ресурс; интерфейс запрашивает подтверждение этой операции.

ЗАКЛЮЧЕНИЕ

Разработано приложение для варианта 2 с регистрацией в PostgreSQL, хранением паролей в виде Argon2id-хэшей и дискреционным управлением ресурсами. Владелец выдаёт и отзывает независимые права чтения, изменения и удаления. Проверка прав выполняется сервером при каждом обращении.

На контейнерном стенде прошли 30 тестов backend и 14 браузерных проверок. Фактически показаны разрешённое чтение, отказ пользователю без права, запрет изменения при наличии только чтения и отказ после отзыва. Сохранены миграции, скриншоты, результаты и инструкции повторного запуска.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лабораторная работа 4. Регистрация пользователя и модели разграничения доступа.
2. Docker Docs: Control startup and shutdown order in Compose [Электронный ресурс]. URL: https://docs.docker.com/compose/how-tos/startup-order/ (дата обращения: 05.11.2026).
3. Alembic: Tutorial [Электронный ресурс]. URL: https://alembic.sqlalchemy.org/en/latest/tutorial.html (дата обращения: 05.11.2026).
4. argon2-cffi: API Reference [Электронный ресурс]. URL: https://argon2-cffi.readthedocs.io/en/stable/api.html (дата обращения: 05.11.2026).
5. FastAPI: OAuth2 with Password and hashing, Bearer with JWT tokens [Электронный ресурс]. URL: https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/ (дата обращения: 05.11.2026).